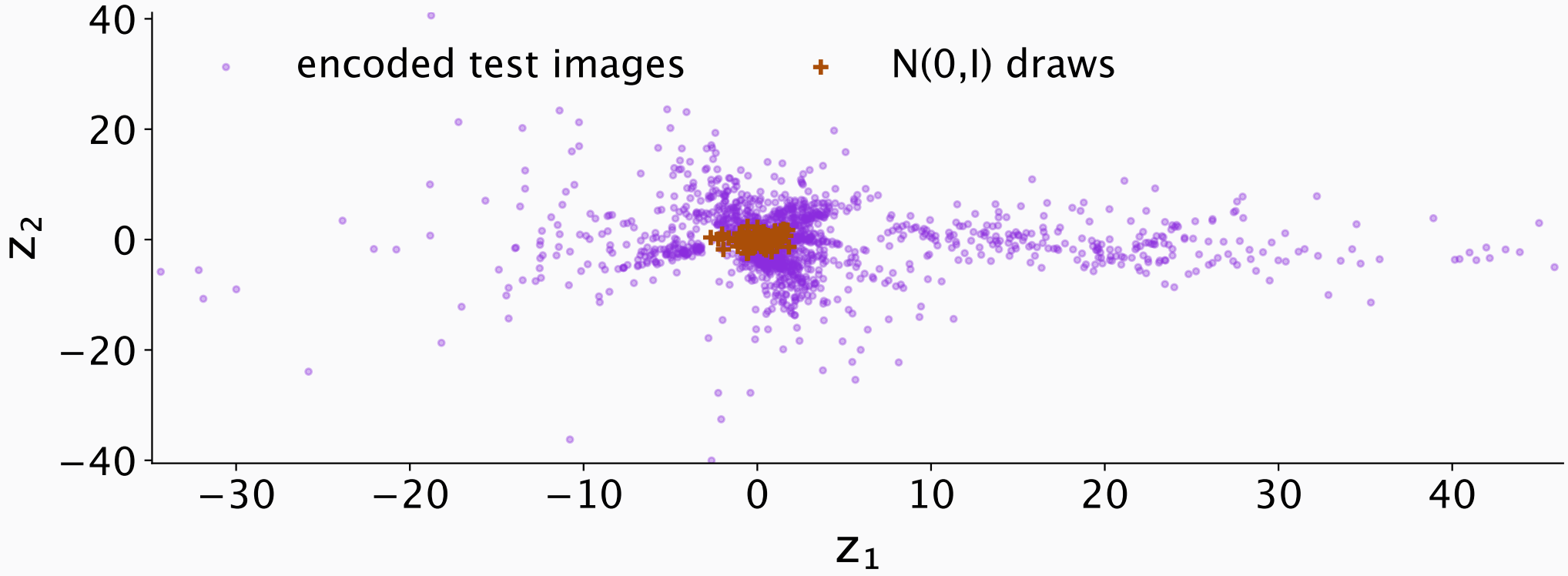
This is exactly where the autoencoder left us



The exact same AE code cloud and N(0,I) draws from Lecture 2.

Reconstruction trained a useful mapping. It did not specify a compatible prior.

> Where standard-normal draws land relative to learned AE codes.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

The same decoder, an unsupported choice of codes



Same fixed AE prior-decoding experiment as Lecture 2.

The failure belongs to this model and this sampling choice; it motivates an explicit generative story.

> Yes. Here we explore learning a probabilistic generative model directly.

Start with a sampling procedure we can name

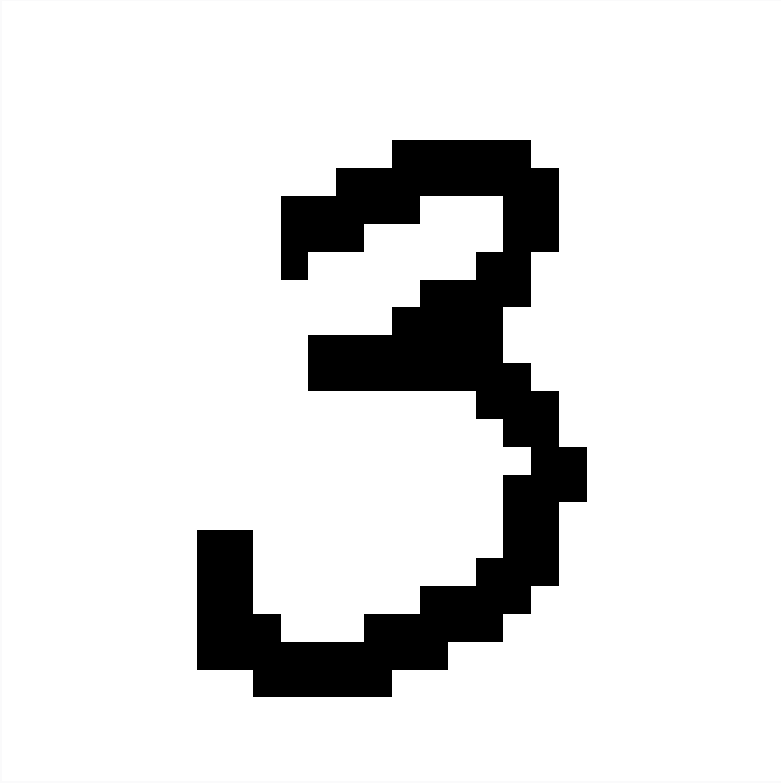
$$z \sim p(z), \qquad x \sim p_\theta(x \mid z)$$

The prior is chosen. The likelihood is learned.

> The prior p(z), without an input image.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

And explain this observed 3



$$p_\theta(z \mid x) = ?$$

Same binary test image, index 90.

A generative model creates an inference question for each observed image.

> Which latent values could have produced this x.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Weights and latent values have different jobs

Symbol	Meaning
x	Observed binary image
z	Latent value for one image
θ	Shared decoder weights
φ	Shared inference-network weights

The fixed standard-normal prior does not acquire learned weights merely because the decoder does.

> No. θ is shared across images.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

The model tells us what inference must solve



We will recover the encoder from the inference problem.

> That would hide the probability model and the reason for the objective.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Choose a fixed prior for latent vectors

$$p(z) = \mathcal{N}(z; 0, I), \qquad z \in \mathbb{R}^d$$

```
z = torch.randn(number_of_images, latent_dim)
```

> Unit variance and zero covariance across latent coordinates.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

The decoder outputs 784 logits



$$a = G_\theta(z), \qquad \pi_j = \mathrm{sigmoid}(a_j)$$

> No. They parameterise probabilities.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Conditioned on z, specify a Bernoulli likelihood

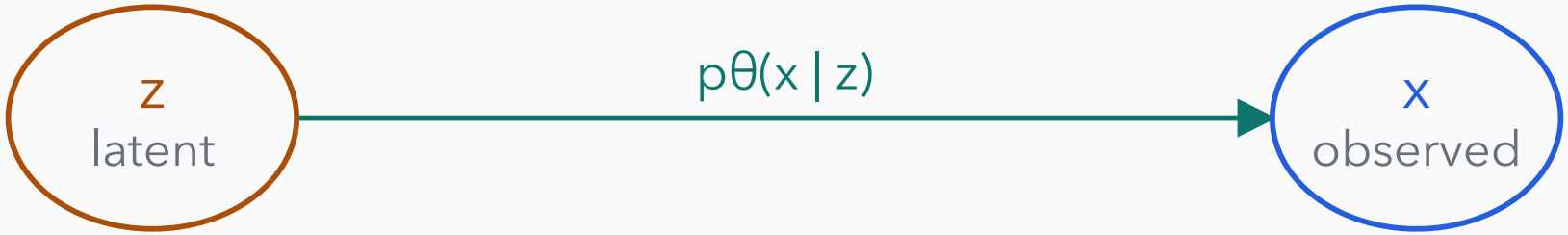
$$p_\theta(x \mid z) = \prod_{j=1}^{784} \mathrm{Bernoulli}(x_j; \pi_j(z))$$

Pixel independence is conditional on the same shared z.

> Yes. The shared latent can coordinate their probabilities.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

A probabilistic graph says who depends on whom



$$p_\theta(x, z) = p(z)\, p_\theta(x \mid z)$$

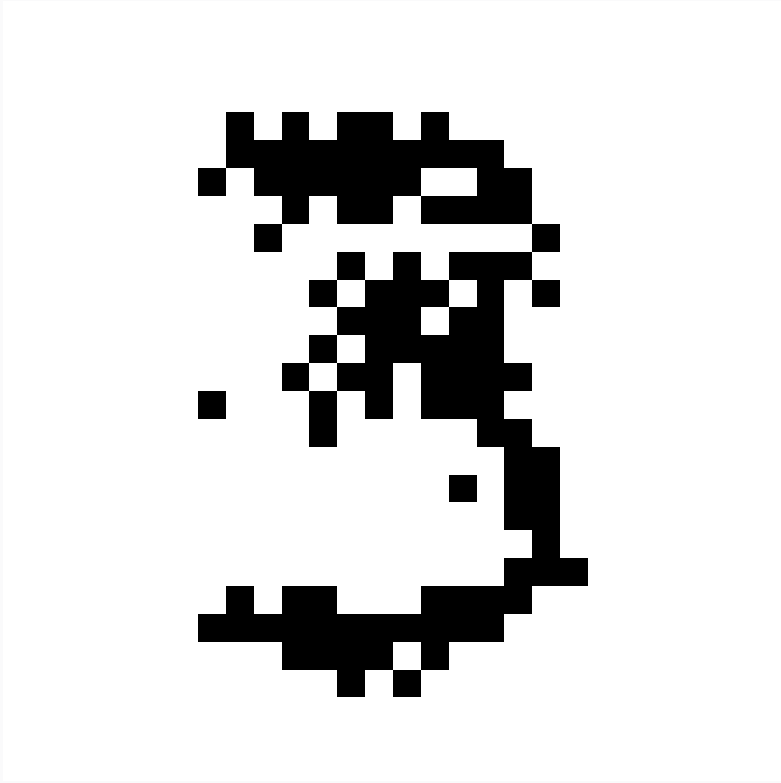
A conditional dependency pθ(x|z), not a layer-by-layer network.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

A probability image and a sampled observation differ



Decoder probability image π(z)



Bernoulli draw x from those probabilities

The probability image can be grey. A draw from this likelihood is binary.

> Bernoulli pixel sampling adds observation randomness.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Two random draws belong to different levels

```
z = torch.randn(10, latent_dim)
logits = decoder(z)
probabilities = logits.sigmoid()
```

```
x = torch.bernoulli(probabilities)
```

> The z draw; the final Bernoulli draw samples pixels given that choice.

Every possible z can contribute to pθ(x)

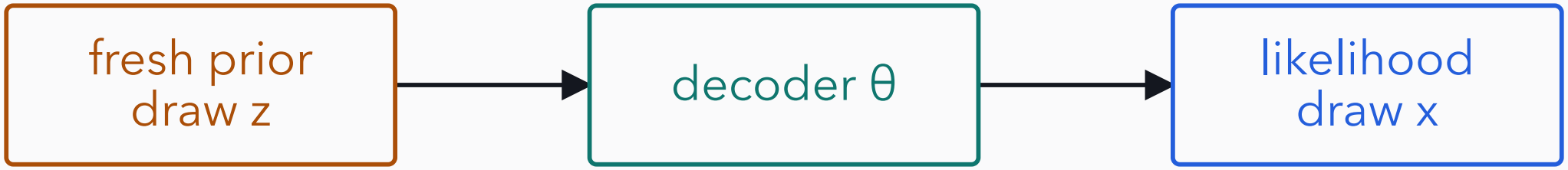
$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz$$

The integral averages over latent explanations under the prior.

> The marginal likelihood sums or integrates all latent explanations.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Generation does not require the encoder

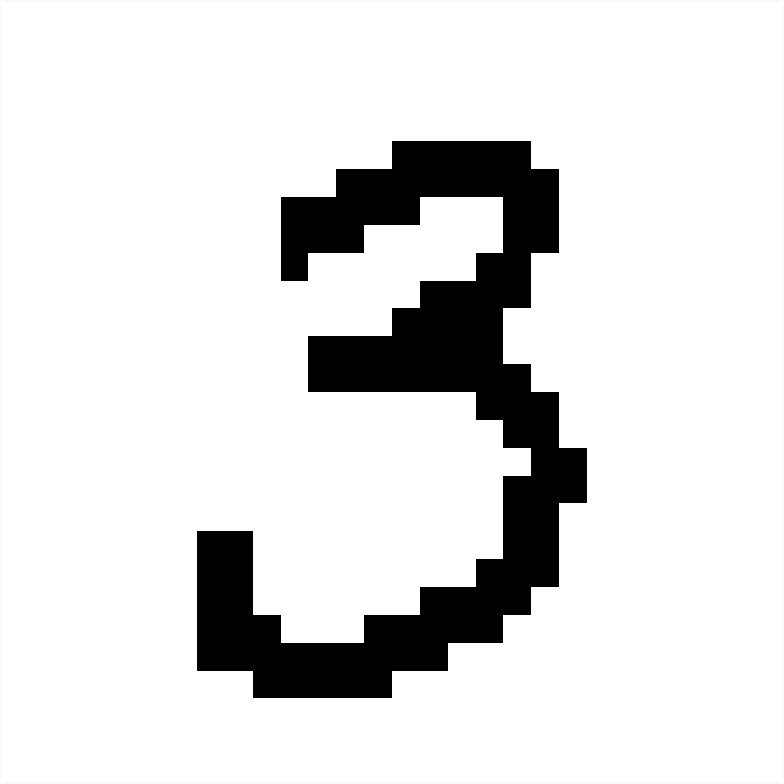


No observed image enters this path.

> To make inference tractable when learning from observed images.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Which latent values explain this image?



$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z)p(z)}{p_\theta(x)}$$

The denominator is the evidence integral from the previous scene.

> Normalising over all z.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Fit a tractable distribution for this one image

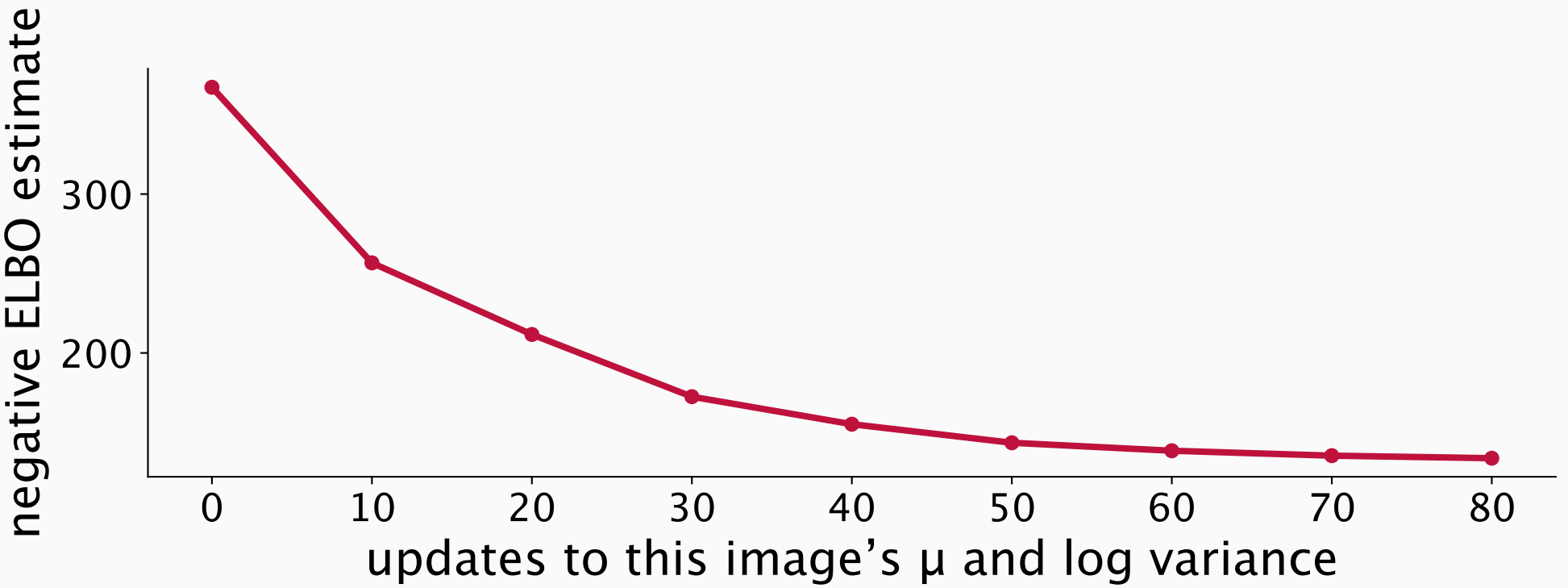
$$q_\psi(z) = \mathcal{N}\left(z; \mu, \mathrm{diag}(\sigma^2)\right)$$

For now: fix θ and optimise this image's free parameters ψ.

> Its μ and log variance, not the fixed decoder weights.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Per-image VI is a real optimisation loop



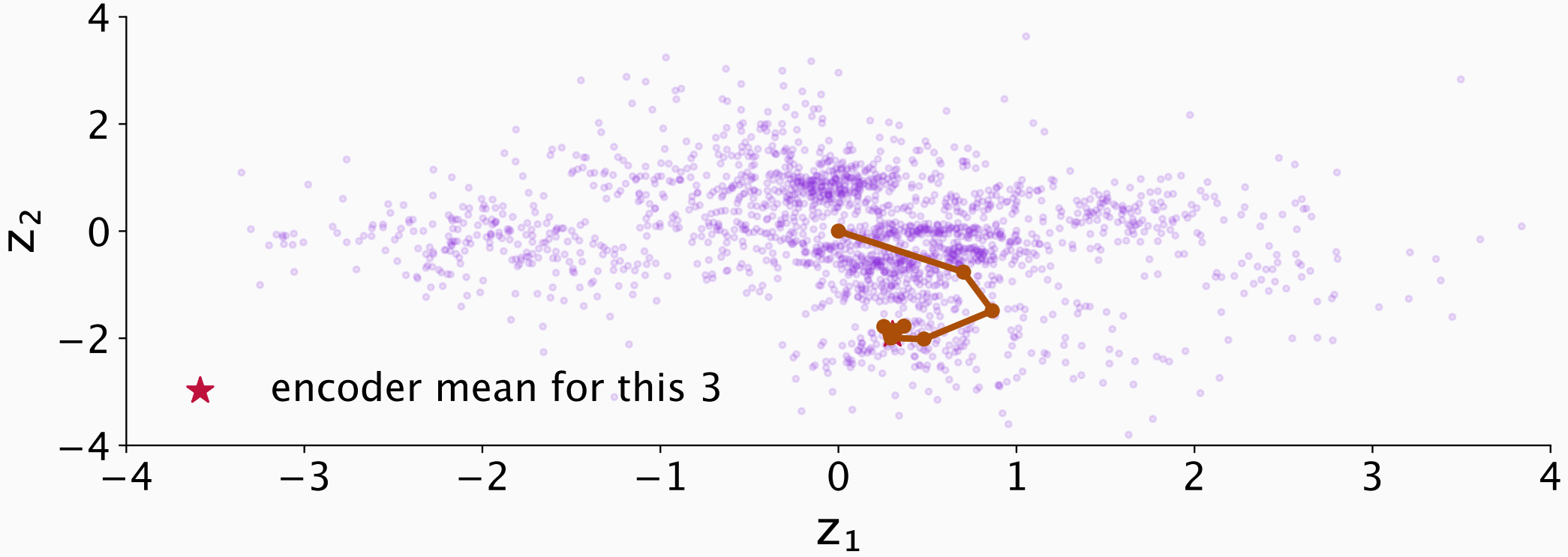
80 actual updates of μ and log variance. Fixed decoder; MC objective estimates.

This demonstrates optimisation, not an exact posterior or certified zero gap.

> The true objective; its Monte Carlo estimate can vary.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

One fit moves its own variational parameters



Amber path: per-image μ. Star: shared encoder's μ for the same 3.

The two procedures need not return the same approximation.

> A new optimisation of that image's q parameters.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Do we run a new optimiser for every image?

Image	Separate inference problem
First 3	Optimise ψ_1
Next 7	Optimise ψ_2
New test image	Start another fit

The decoder is shared; the variational parameters are solved separately.

> A mapping from an image to useful variational parameters.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Learn to predict the variational parameters



$$q_\phi(z \mid x) = \mathcal{N}(z; \mu_\phi(x), \mathrm{diag}(\sigma^2_\phi(x)))$$

The network weights φ; its output parameters depend on x.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Amortisation trades repeated solving for a shared map

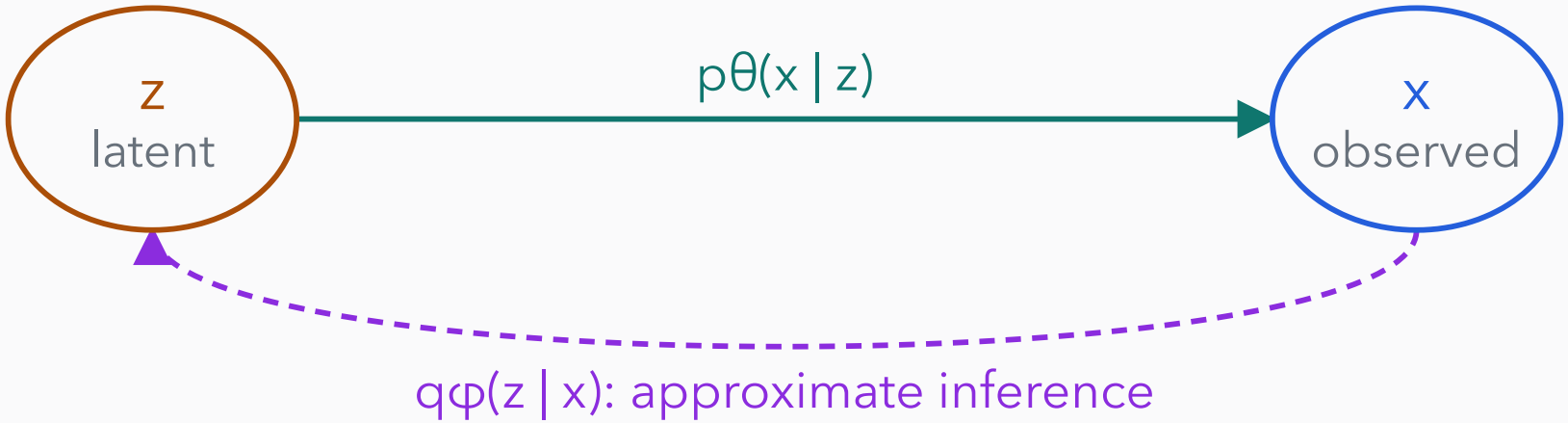
Separate VI	Amortised VI
Free ψ for each image	One shared φ
Optimisation for each new image	One encoder evaluation
Chosen family still restricts q	Family and shared mapping restrict q

The encoder is an inference network because it predicts an approximate posterior.

> No. It is a learned approximation.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

The model and inference answer opposite questions



Solid: generate x from z. Dashed: approximate the posterior of z after observing x.

> No. It labels the approximate-inference relationship.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Reuse the identity, with x fixed

$$\log p_\theta(x) = \mathcal{L}(\theta, \phi; x) + \mathrm{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x))$$

For fixed θ, raising the ELBO lowers the posterior gap.

> The model and its evidence change; the total is no longer fixed across updates.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

The joint-density form remains the starting point

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x, z) - \log q_\phi(z \mid x)]$$

$$\log p_\theta(x, z) = \log p_\theta(x \mid z) + \log p(z)$$

> The factorisation into likelihood and prior.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Split likelihood from the prior-to-q comparison

$$\mathcal{L} = \mathbb{E}_q[\log p_\theta(x \mid z)] + \mathbb{E}_q[\log p(z) - \log q(z \mid x)]$$

$$\mathcal{L} = \mathbb{E}_q[\log p_\theta(x \mid z)] - \mathrm{KL}(q_\phi(z \mid x) \| p(z))$$

> Its log ratio is the negative of log q minus log p.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

These two KL terms compare different distributions

Term	What it compares
Posterior gap	$q_\varphi(z\|x)$ versus $p_\theta(z\|x)$
Computable prior KL	$q_\varphi(z\|x)$ versus $p(z)$

The prior KL is part of the objective. It is not the unknown posterior gap.

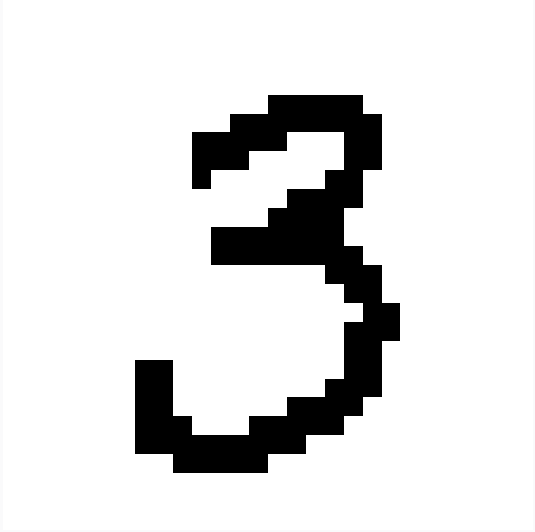
> No. q then equals the prior, which need not be this image's posterior.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

A sampled z should explain the observed pixels

$$-\log p_\theta(x \mid z) = \sum_{j=1}^{784} \ell_j$$

Each ℓj is the Bernoulli negative log-likelihood for the observed pixel.



Target x



Probability image at one q draw

> It selects the observed Bernoulli outcomes for all pixels.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

The diagonal-Gaussian prior KL is available exactly

$$\mathrm{KL}(q\|p) = \frac{1}{2}\sum_{j=1}^{d}\left(\mu_j^2 + \sigma_j^2 - 1 - \log\sigma_j^2\right)$$

Here p=N(0,I), and q has diagonal covariance. This formula is not universal.

> μ=0 and σ²=1 in every coordinate.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

The formula penalises shifted or mismatched uncertainty

$$\frac{1}{2}\mu_j^2 \quad + \quad \frac{1}{2}(\sigma_j^2 - 1 - \log \sigma_j^2)$$

The mean term vanishes at zero; the scale term vanishes at unit variance.

> No. The −log σ² term grows without bound.

For this 3, the two contributions are measured

Quantity	Value in nats
One-draw reconstruction NLL	122.262
Exact prior KL	6.966
One-draw negative ELBO estimate	129.228

The reconstruction term uses $\varepsilon=(0.62,-1.15)$. The KL term uses no sampling estimate.

> The reconstruction term and total estimate; the analytic KL stays fixed.

CPU replay of vae2, official test90, stated chosen ε; fixture full precision, displayed rounding.

Maximise the bound, or minimise its negative

$$\max_{\theta,\phi} \ \mathcal{L}$$

$$\min_{\theta,\phi} \ \underbrace{-\mathbb{E}_q \log p_\theta(x \mid z)}_{\text{reconstruction NLL}} + \underbrace{\mathrm{KL}(q \| p)}_{\text{prior KL}}$$

No. It uses the likelihood, q and the prior.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

The reduction is part of the model implementation

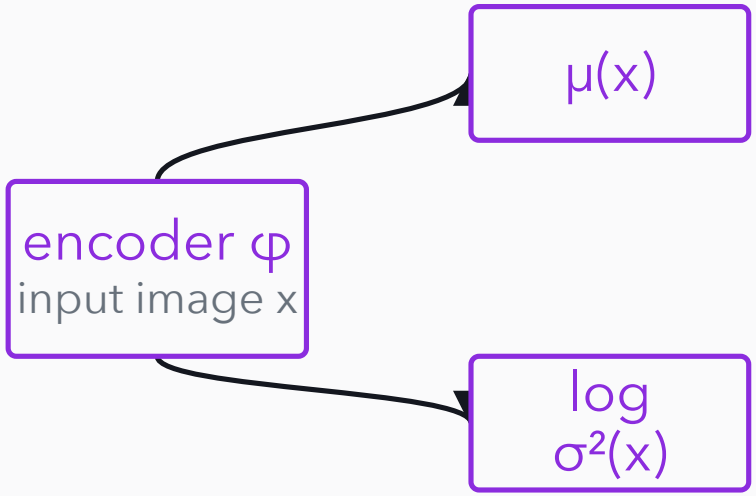
$$L = \frac{1}{B}\sum_{i=1}^{B}\left[\sum_{j=1}^{784}\ell_{ij} + \frac{1}{2}\sum_{k=1}^{d}(\mu_{ik}^2 + e^{v_{ik}} - 1 - v_{ik})\right]$$

v means log variance. Sum event dimensions; average examples.

> No. It would change their relative scale by 784.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

The encoder predicts two vectors



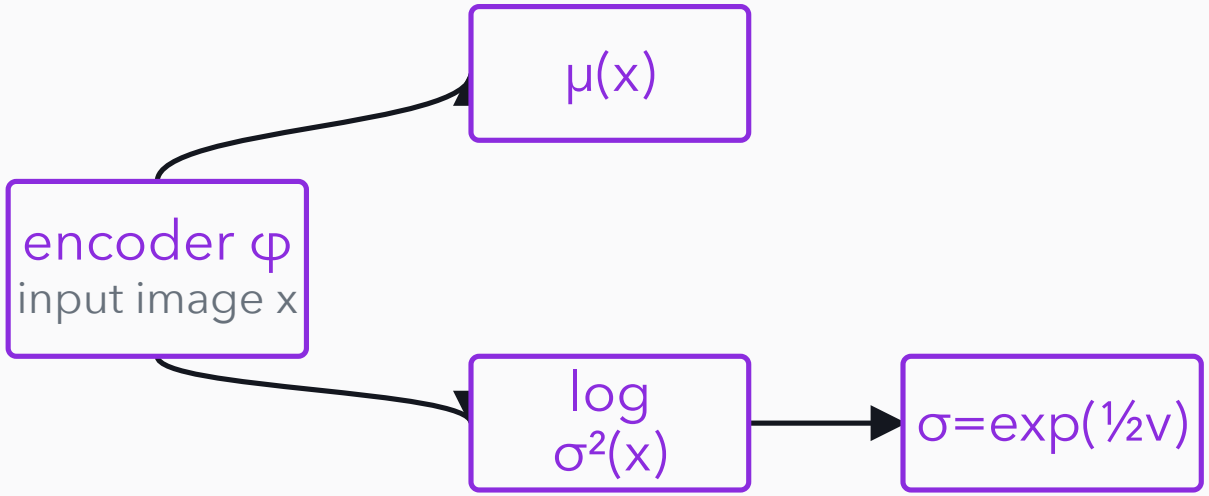
One head predicts μ; another predicts v=log σ². Neither is a random draw.

> Both μ and log variance.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Half the log variance before exponentiating

$$v = \log \sigma^2 \quad \Longrightarrow \quad \sigma = \exp(v/2)$$



> The variance, not the standard deviation.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Our 3 produces a narrow Gaussian

Vector	Coordinate 1	Coordinate 2
μ	0.304	-1.932
$\log \sigma^2$	-6.106	-5.997
σ	0.0472	0.0499

These outputs describe $q_\varphi(z|x)$, not the prior $N(0,I)$.

> No. It is the approximation for this particular image.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Draw noise that does not depend on φ

$$\epsilon \sim \mathcal{N}(0, I)$$

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$$

> Through μ and σ, while ε is an independent base draw.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Calculate this particular latent draw

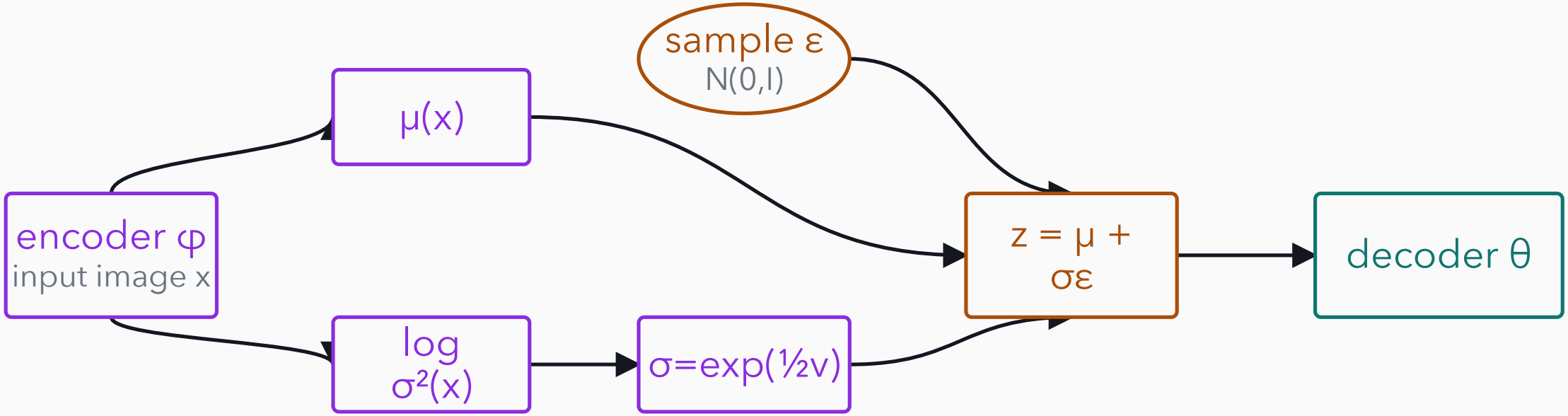
$$z \approx \begin{bmatrix} 0.304 \\ -1.932 \end{bmatrix} + \begin{bmatrix} 0.047 \\ 0.050 \end{bmatrix} \odot \begin{bmatrix} 0.620 \\ -1.150 \end{bmatrix}$$

$$z \approx \begin{bmatrix} 0.333 \\ -1.990 \end{bmatrix}$$

> Compare σjɛj; the second displacement has larger magnitude here.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Randomness enters through one explicit input



For a fixed ε, every operation from μ and v to the decoder is differentiable.

> The dependence of sampled z on the encoder outputs.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Follow one coordinate by hand

$$z = \mu + e^{v/2}\epsilon$$

$$\frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial v} = \frac{1}{2}e^{v/2}\epsilon$$

Differentiating exp(v/2) supplies it.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

A gradient check keeps the same noise

```
loss_plus  = loss(mu + h, fixed_epsilon)
loss_minus = loss(mu - h, fixed_epsilon)
finite_difference = (loss_plus-loss_minus)/(2*h)
```

Use the same ε on both sides. Training normally draws fresh noise each batch.

> The finite difference would mix parameter change with sampling noise.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Reparameterisation supplies the pathwise gradient

A detached sample can hide the dependence of z on φ, even if another part of the loss still returns a gradient.

Other gradient estimators exist. Here we use the Gaussian location-scale path because it is simple and differentiable.

> No. Inspect which paths the computation graph includes.

The forward pass has one source of latent noise

```
mu, logvar = encoder(x)
std = torch.exp(0.5 * logvar)
epsilon = torch.randn_like(std)
```

```
z = mu + std * epsilon
logits = decoder(z)
```

μ, logvar, std, ε and z. Logits have shape B×784.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Keep the event sum visible

```
pixel_nll = F.binary_cross_entropy_with_logits(
    logits, x.flatten(1), reduction="none")
```

```
recon_nll = pixel_nll.sum(dim=1)
```

B values, one reconstruction NLL per image.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

The analytic KL has the same outer shape

```
kl_per_dimension = 0.5 * (
    mu.square() + logvar.exp() - 1 - logvar)
```

```
prior_kl = kl_per_dimension.sum(dim=1)
```

> The diagonal-Gaussian KL adds coordinate contributions.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

One complete optimisation step

```
loss = (recon_nll + prior_kl).mean()
optimizer.zero_grad(set_to_none=True)
loss.backward()
```

```
optimizer.step()
```

> Both decoder θ and encoder φ, including both encoder heads.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Check a real update, not only a loss formula

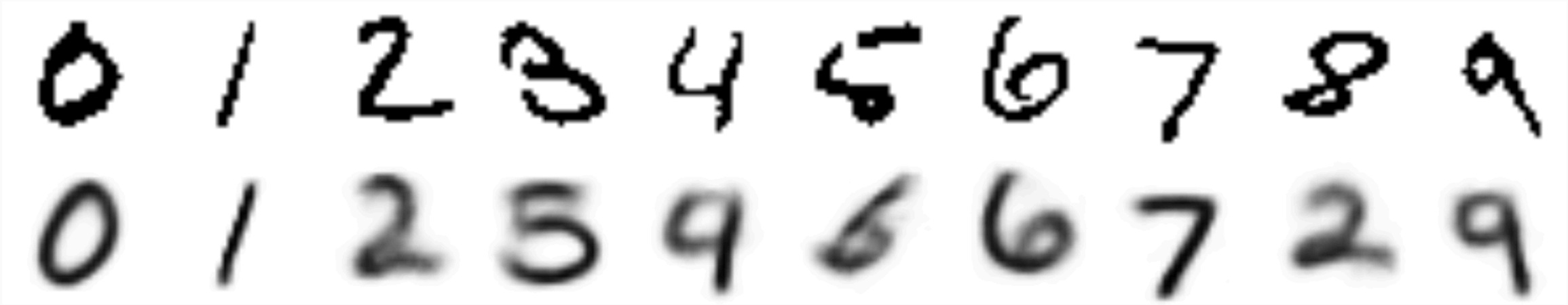
Measured quantity	Value
Training batch	8 binary MNIST images
SGD step size	$\eta = 0.000001$
Same-noise loss before	126.017
Same-noise loss after	125.918

Both encoder heads and decoder have nonzero gradients and parameter changes.

> It isolates the parameter update from sampling variation.

Actual copied-checkpoint SGD execution; fixed-noise diagnostic, not a claim every stochastic training step must decrease loss.

Inspect the existing trained model on held-out images



Top: observed test images. Bottom: decode the encoder means; d=2.

Mean-code reconstruction is a useful visual diagnostic, not an estimate of expected reconstruction loss.

> A nonlinear decoder generally maps the mean differently from the mean of its outputs.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Draw z from the prior and decode



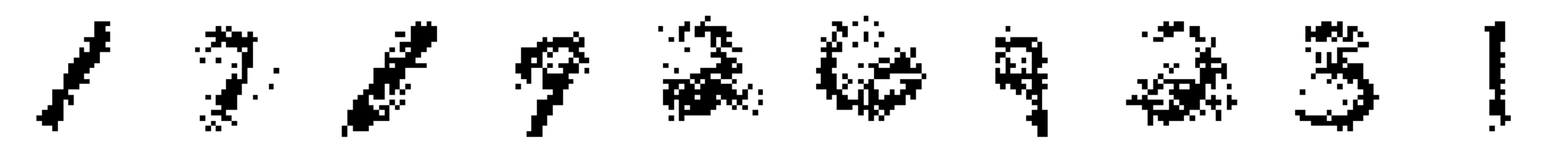
Ten fixed prior draws, d=2. Displayed outputs are Bernoulli probabilities.

The encoder is absent from this computation.

> Only the decoder θ; the prior is fixed.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Sample observations from those same likelihoods



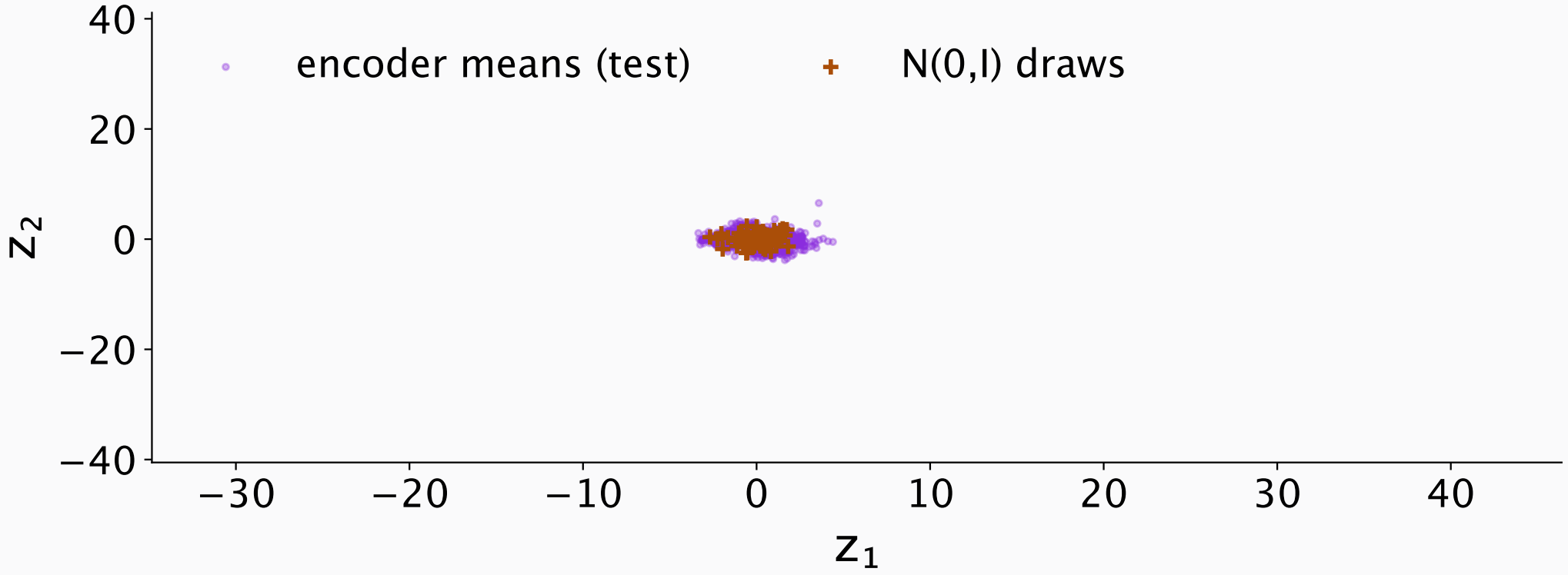
Bernoulli observations using the same ten latent draws.

Use “probability image” or “sampled observation” precisely in every caption.

> Only the Bernoulli observation draws; z stayed fixed.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Reuse the AE axes for the VAE comparison



Purple: encoder means for test images. Amber +: the same prior draws. AE comparison axes retained.

This shows encoder means, not samples from the aggregate posterior.

> No. Each q also has a covariance; this plot alone cannot establish equality.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

A larger latent changes capacity, not the objective’s meaning



d=16: mean-code reconstructions



d=16: prior probability images

These are recorded checkpoints, trained for their own stated schedules.

> No. Comparisons need the same observations and reduction convention.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

We now have both a generative model and fast inference

Operation	Computation
Generate	$z \sim p(z)$, then $x \sim p_\theta(x \mid z)$
Infer	$q_\varphi(z \mid x)$ from the encoder
Learn	Optimise a computable lower bound

The encoder solves an approximate-inference task needed during learning.

> A specified probabilistic generative model and a variational training objective.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

A chosen prior is not a guarantee of perfect samples

The decoder, variational family, optimisation and data coverage can all be imperfect.

Inspect generated images and relevant diagnostics; a training objective alone does not certify quality.

> No such guarantee follows from the objective.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

A stable ELBO does not certify exact inference

$$\log p_\theta(x) - \mathcal{L} = \mathrm{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x))$$

Family restriction, amortisation and incomplete optimisation can leave a gap.

> The exact posterior KL or exact log evidence, generally unavailable here.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Setting the KL weight to zero does not remove sampling

$$L_{\beta_{\mathrm{KL}}} = \text{reconstruction NLL} + \beta_{\mathrm{KL}}\,\mathrm{KL}(q\|p)$$

At βKL=0, z=μ+σε still samples unless we explicitly change that computation.

> No. The stochastic encoder and likelihood computation can remain.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Explain what each path needs

Task	Observed x?	Encoder?
Training from data	Yes	Yes
Approximate inference	Yes	Yes
Prior generation	No	No

The decoder is shared; its role depends on which latent source is used.

> Training; prior generation does not have a reconstruction target.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Could a learned critic supply the training signal?

The VAE used a likelihood and a variational bound. Next we will train a generator with an adversarial classification signal.

Keep the same questions: where does randomness enter, what is learned, and what does the evidence support?

> Explicit sampling path, parameter owners, gradient path and honest sample-quality claims.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Optional: a controlled change in KL weighting



Recorded d=2 βKL=8 prior probability images; same fixed prior draws.

Recorded model	Reconstruction estimate	Prior KL
βKL=1	129.005	7.076
βKL=8	139.916	3.869

> Compare the measured reconstruction and KL values; do not claim a universal monotone sample-quality trend.

Existing d2 β1/β8 checkpoints: same architecture, seed and 40-epoch schedule; held-out one-draw replay seed673. Different objectives.

Optional: mean clouds omit posterior spread

$$q_\phi(z) = \frac{1}{N} \sum_{i=1}^{N} q_\phi(z \mid x_i)$$

This mixture over images is an aggregate posterior. A scatter of μφ(xi) draws only its component centres.

> The covariance and probability mass around each centre.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.

Optional: when is log evidence a fixed total?

During this update	Does log pθ(x) change?
Only φ, fixed θ and x	No
θ changes	Generally yes

The identity holds in both cases. The fixed-total picture only applies when θ and x are fixed.

> The quantities held fixed differ across the two optimisation problems.

Original teaching construction; exact formula or measured checkpoint replay. See data/provenance.json and fixtures.json.